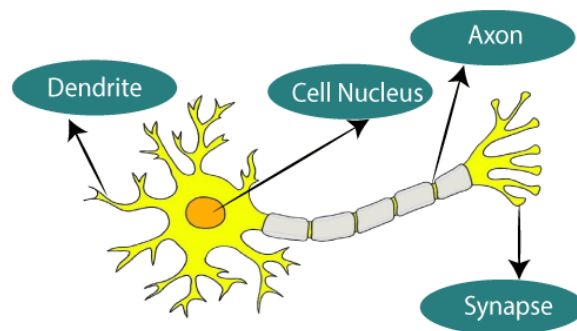


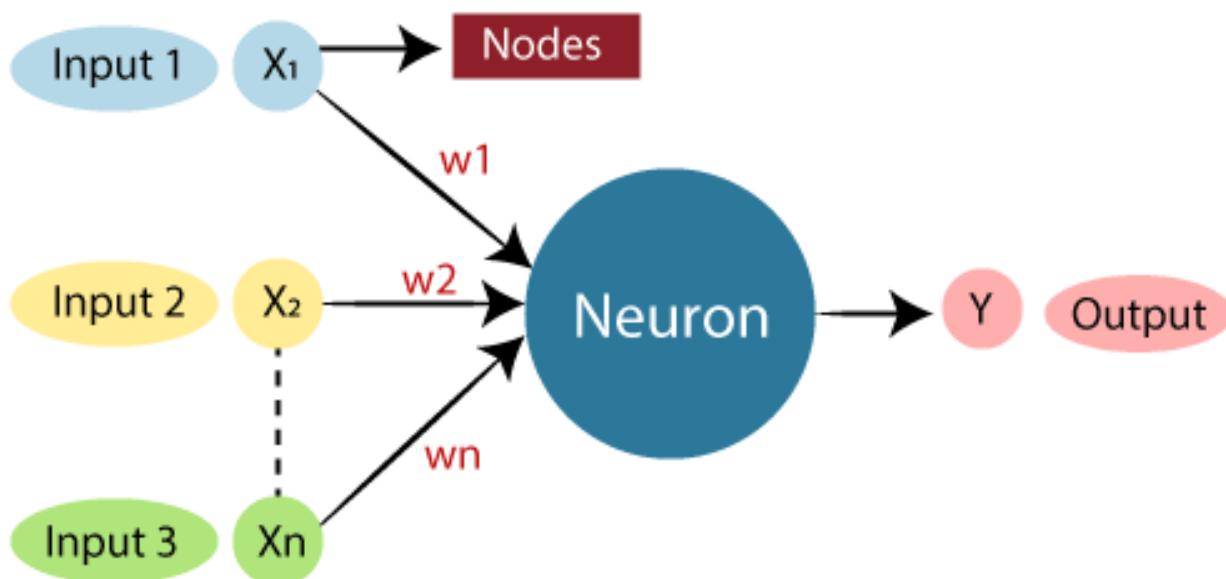
Unit-III

1. What is an artificial neural network (ANN)? Describe its basic structure and components.

- ❖ The term "Artificial Neural Network" is derived from Biological neural networks that develop the structure of a human brain.
- ❖ Similar to the human brain that has neurons interconnected to one another, artificial neural networks also have neurons that are interconnected to one another in various layers of the networks. These neurons are known as nodes.
- ❖ The given figure illustrates the typical diagram of Biological Neural Network.



- ❖ The typical Artificial Neural Network looks something like the given figure



- ❖ Dendrites from Biological Neural Network represent inputs in Artificial Neural Networks, cell nucleus represents Nodes, synapse represents Weights, and Axon represents Output.
- ❖ Relationship between Biological neural network and artificial neural network:

Biological Neural Network

Dendrites

Cell nucleus

Synapse

Axon

Artificial Neural Network

Inputs

Nodes

Weights

Output

- ❖ An Artificial Neural Network in the field of Artificial intelligence where it attempts to mimic the network of neurons makes up a human brain so that computers will have an option to understand things and make decisions in a human-like manner.
- ❖ **The artificial neural network is designed by programming computers to behave simply like interconnected brain cells.**
- ❖ There are around 1000 billion neurons in the human brain.
- ❖ Each neuron has an association point somewhere in the range of 1,000 and 100,000.
- ❖ In the human brain, data is stored in such a manner as to be distributed, and we can extract more than one piece of this data when necessary from our memory parallelly.
- ❖ We can say that the human brain is made up of extraordinarily **amazing parallel processors**.

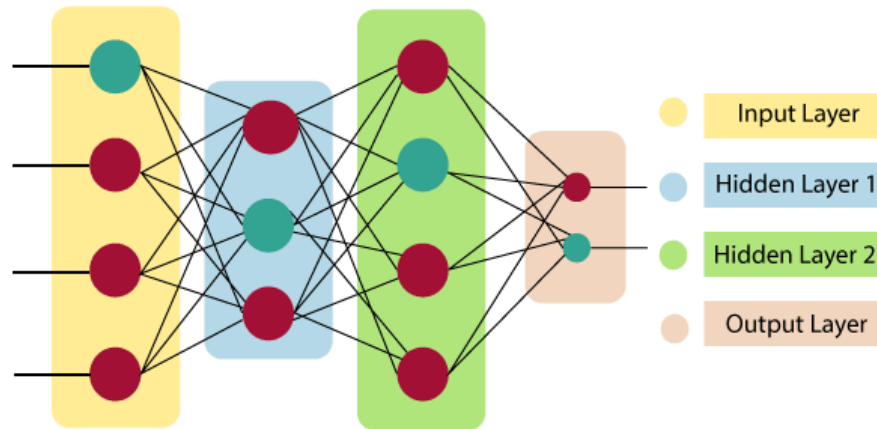
The architecture of an artificial neural network:

- ❖ To understand the concept of the architecture of an artificial neural network, we have to understand what a neural network consists of. In order to define a neural network that consists

of a large number of artificial neurons, which are termed units arranged in a sequence of layers.

Lets us look at various types of layers available in an artificial neural network.

- ❖ Artificial Neural Network primarily consists of three layers:



Input Layer:

- ❖ As the name suggests, it accepts inputs in several different formats provided by the programmer.

Hidden Layer:

- ❖ The hidden layer presents in-between input and output layers. It performs all the calculations to find hidden features and patterns.

Output Layer:

- ❖ The input goes through a series of transformations using the hidden layer, which finally results in output that is conveyed using this layer.
- ❖ The artificial neural network takes input and computes the weighted sum of the inputs and includes a bias. This computation is represented in the form of a transfer function.

$$\sum_{i=1}^n w_i * X_i + b$$

- ❖ It determines weighted total is passed as an input to an activation function to produce the output. Activation functions choose whether a node should fire or not.
- ❖ Only those who are fired make it to the output layer.
- ❖ There are distinctive activation functions available that can be applied upon the sort of task we are performing.

Advantages of Artificial Neural Network (ANN)

Parallel processing capability:

- ❖ Artificial neural networks have a numerical value that can perform more than one task simultaneously.

Storing data on the entire network:

- ❖ Data that is used in traditional programming is stored on the whole network, not on a database. The disappearance of a couple of pieces of data in one place doesn't prevent the network from working.

Capability to work with incomplete knowledge:

- ❖ After ANN training, the information may produce output even with inadequate data. The loss of performance here relies upon the significance of missing data.

Having a memory distribution:

- ❖ For ANN is to be able to adapt, it is important to determine the examples and to encourage the network according to the desired output by demonstrating these examples to the network.
- ❖ The succession of the network is directly proportional to the chosen instances, and if the event can't appear to the network in all its aspects, it can produce false output.

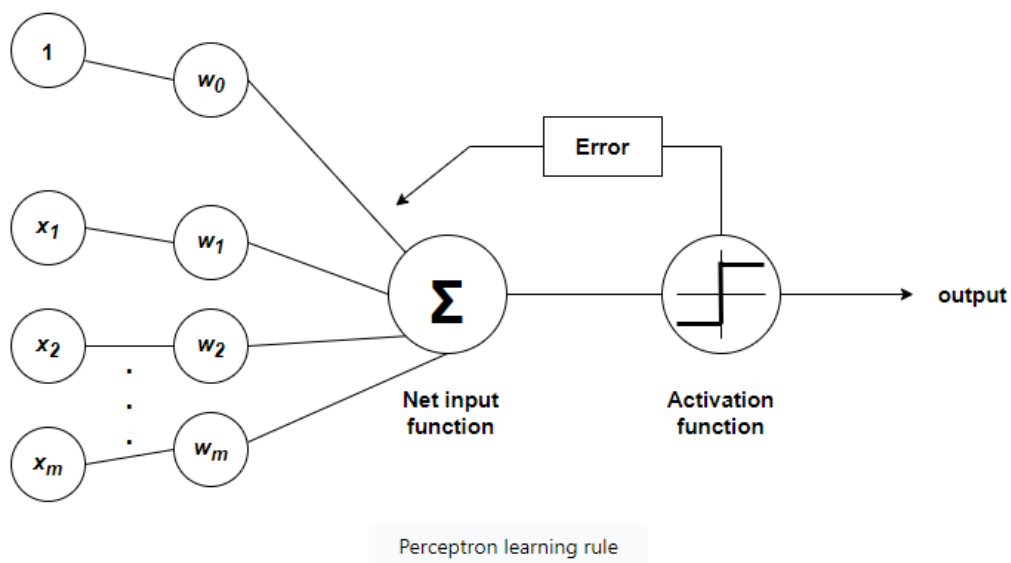
Having fault tolerance:

- ❖ Extortion of one or more cells of ANN does not prohibit it from generating output, and this feature makes the network fault-tolerance.

2. Explain Perceptron Learning Rule, How does it work?

Perceptron learning rule

- ❖ A perceptron takes inputs, modifies the inputs using certain weights, and then uses a function to output the final result. This is shown in the diagram below



The output generated is based on the input values, $x_1, x_2, \dots, x_m$. The output can only have two values (binary classification), usually 1 or 0.

The summation function (represented by Σ in the above diagram) multiplies the inputs with the weights and then adds them up. This can be represented using the following equation:

$$w_0 + w_1x_1 + w_2x_2 + \dots + w_mx_m$$

$$o(x_1, x_2, x_3, \dots, x_m) = \begin{cases} 1 & \text{if } w_0 + w_1x_1 + w_2x_2 + \dots + w_mx_m > 0 \\ -1 & \text{otherwise} \end{cases}$$

A simple activation function

Activation function

- ❖ The activation function converts the numerical output of the weighting function to 1 or 0.
- ❖ The following is an example of a simple activation function:

Error and adjustments

- ❖ Before we can use a perceptron to predict output, it is trained using labeled data. For each input in the training set, we compute the output. If the observed output does not match the expected output, we calculate the error.
- ❖ Initially, we usually set the weights to randomly selected numbers.
- ❖ We can then use the error to tweak the weights in favor of the expected output. We repeat this process until the perceptron gives a high degree of accuracy in its output.
- ❖ How much we adjust our weights is controlled by the learning rate of the perceptron.

Perceptron Training Algorithm

- ❖ To find a set of weights and a bias that allows a perceptron to correctly classify a set of linearly separable input patterns into two classes.

Inputs:

- ❖ A set of training examples: $\{(x_1, t_1), (x_2, t_2), \dots, (x_P, t_P)\}$

- ❖ x_p : An input vector for the p -th example (e.g., $[x_{p1}, x_{p2}, \dots, x_{pN}]$, where N is the number of input features).
- ❖ t_p : The target output (desired class label) for the p -th example, typically -1 or 1 for binary classification.
- ❖ Learning rate η (eta): A small positive constant, usually between 0.0 and 1.0 (e.g., 0.1).

Outputs:

- ❖ A set of optimal weights $w = [w_1, w_2, \dots, w_N]$
- ❖ An optimal bias b

Algorithm Steps:

Step 1: Initialization

- ❖ **Initialize Weights (w):** Set all connection weights w_i (for $i = 1$ to N) to small random values, often 0. Alternatively, you can initialize them to all zeros. For simplicity in theoretical examples, all zeros is common.
- ❖ **Initialize Bias (b):** Set the bias b to a small random value, often 0.
- ❖ **Set Learning Rate (η):** Choose a suitable learning rate, e.g., $\eta = 0.1$.

Step 2: Define Activation Function (Activation Rule)

- ❖ The perceptron typically uses a step (or sign/bipolar) activation function. For a calculated net input S :
 - ❖ If $S \geq 0$, then the output $y = 1$.
 - ❖ If $S < 0$, then the output $y = -1$.

Step 3: Training Loop (Epochs)

- ❖ Loop: Repeat the following steps for a fixed number of epochs (iterations over the entire training dataset) or until the network converges (no misclassifications occur for an entire epoch).

Step 4: Iterate Through Training Examples

- ❖ For each training example p from 1 to P :
 - ❖ Let the current input vector be $x_p = [x_{p1}, x_{p2}, \dots, x_{pN}]$.

- ❖ Let the current target output be t_p .

Step 5: Calculate Net Input (S)

- ❖ Compute the net input (or weighted sum) for the perceptron's output neuron:
- ❖ $S = b + (x_{p1} * w_1) + (x_{p2} * w_2) + \dots + (x_{pN} * w_N)$
- ❖ This can be written in vector form as:
- ❖ $S = b + x_p \cdot w$ (where $\cdot$ denotes the dot product)

Step 6: Calculate Actual Output (y)

- ❖ Apply the activation function (from Step 2) to the net input S to get the perceptron's actual output y:
- ❖ If $S \geq 0$, then $y = 1$.
- ❖ If $S < 0$, then $y = -1$.

Step 7: Update Weights and Bias (Perceptron Learning Rule)

- ❖ **Check for Error:** Compare the actual output y with the target output t_p .
- ❖ If $y \neq t_p$ (Error occurred):
 - ❖ **Calculate Error Term (δ):** $\delta = t_p - y$
 - ❖ Note: If $t_p = 1$ and $y = -1$, $\delta = 1 - (-1) = 2$.
 - ❖ If $t_p = -1$ and $y = 1$, $\delta = -1 - 1 = -2$.
 - ❖ **Update Weights:** For each weight w_i :
 - ❖ $w_{i(new)} = w_{i(old)} + \eta * \delta * x_{pi}$
 - ❖ **Update Bias:**
 - ❖ $b(new) = b(old) + \eta * \delta$
- ❖ **If $y == t_p$ (No Error):**
 - ❖ No update is needed. The weights and bias remain the same for this training example.
 - ❖ $w_{i(new)} = w_{i(old)}$
 - ❖ $b(new) = b(old)$

Step 8: Check for Convergence

- ❖ After iterating through all training examples in an epoch, check if any weights or bias were updated.
- ❖ If no updates occurred during an entire epoch, it means the perceptron has learned all patterns correctly and has converged. The training can stop.
- ❖ Otherwise, go back to Step 3 for the next epoch.

Step 9: Termination

- ❖ The algorithm terminates when:
 - ❖ The perceptron converges (no errors for an entire epoch), or
 - ❖ A predefined maximum number of epochs is reached (to prevent infinite loops in non-linearly separable cases).

3. Explain about Cross Validation in ANN

- ❖ Cross-validation is a model evaluation technique. It is used to assess how well your ANN model will **generalize to an independent dataset** (i.e., data it hasn't seen before). Instead of training the model once and testing it on a single test set, we split the data multiple times and train/test the model multiple times.
- ❖ Cross validation is a resampling procedure used to assess a model's ability to predict new, unseen data. It is especially important in neural networks, as these models are prone to overfitting due to their high capacity and flexibility.

Why is Cross-Validation Needed in ANN?

- ❖ Training neural networks typically requires a lot of data. If we train on one part and test on another, our results might depend too much on how the data was split. Cross-validation:
- ❖ Reduces bias in model evaluation.
- ❖ Gives a better estimate of how the ANN performs.
- ❖ Helps prevent overfitting (learning the training data too well but failing on new data).
- ❖ Helps in hyperparameter tuning (e.g., learning rate, batch size, number of neurons).

How Cross Validation Works in ANN

- ❖ **Data Splitting:**

- ❖ The data is divided into 'k' roughly equal-sized folds or subsets.
- ❖ The model is trained k times: each time, one fold is used for validation, and the remaining (k-1) are used for training.
- ❖ **Training and Evaluation:**
 - ❖ For each fold, a fresh copy of the ANN is initialized and trained.
 - ❖ After training, performance is measured on the validation fold.
 - ❖ This process is repeated for each fold, ensuring each subset gets used for validation once.
- ❖ **Result Aggregation:**
 - ❖ The final performance metric (accuracy, loss, etc.) is calculated by averaging metrics across all k validation folds.
 - ❖ This gives a more robust estimate than a single train/test split.

Types of Cross Validation Used in ANN

- ❖ **K-Fold Cross-Validation (Most Common):**
 - ❖ K-fold cross-validation is a technique used in ML / Deep Learning to evaluate the performance of a model by **dividing the dataset into K subsets (or folds)**, training the model on K-1 folds, and testing it on the remaining fold.
 - ❖ This process is repeated K times, with each fold used as the test set once. The final performance metric is the average of the results from all K iterations, providing a more robust estimate of the model's generalization ability than a single train/test split.

Steps:

1. Divide your data into K equal parts (folds).
2. For each iteration:
 1. Use K-1 folds for training.
 2. Use the remaining 1 fold for validation.
3. Repeat this K times (each fold gets a chance to be the validation set).
4. Average the results (accuracy, loss, etc.).

Example (K=5):

- ❖ Split data into 5 parts: A, B, C, D, E
- ❖ Iteration 1: Train on B+C+D+E, validate on A
- ❖ Iteration 2: Train on A+C+D+E, validate on B
- ❖ and so on.
- ❖ Compute average validation performance.
- ❖ **Advantage:** All data is used for both training and validation.

Benefits of K-fold cross-validation:**Reduced Variance:**

- ❖ By using all data for both training and validation, it provides a more reliable estimate of the model's performance than a single train/test split.

Handles Imbalanced Data:

- ❖ It can be particularly useful when dealing with imbalanced datasets, where one class has significantly fewer instances than others, by ensuring each fold has a representative sample of all classes.

Stratified K-Fold Cross-Validation

- ❖ Stratified K-Fold Cross-Validation is an enhanced version of K-Fold Cross-Validation, particularly valuable for classification problems, especially those with imbalanced datasets.
- ❖ Especially important if classes are imbalanced (e.g., 90% class A, 10% class B).

Leave-One-Out Cross-Validation (LOOCV)

- ❖ Leave-One-Out Cross-Validation (LOOCV) is a specific type of k-fold cross-validation where the number of folds, 'k', is equal to the number of data points, 'n', in the dataset.
- ❖ Extreme case of K-Fold where $K = \text{number of data points}$.
- ❖ Each sample is used once as the validation data, all others are used for training.
- ❖ Very computationally expensive for ANN, rarely used.

Repeated K-Fold Cross-Validation:

- ❖ Repeated k-fold cross-validation is an extension of the standard k-fold cross-validation technique used for evaluating machine learning models. It involves performing the k-fold cross-validation procedure multiple times, with a different random shuffling of the data and subsequent splitting into k-folds for each repetition.
- ❖ Repeat the K-Fold process multiple times (with different splits).
- ❖ Reduces variance of the validation estimate even more.

4. Write about Feature Selections in ANN.

- ❖ Feature Selection is the process of selecting the most important input variables (features) from a dataset that are most relevant to the prediction or classification task. In Artificial Neural Networks (ANNs), good feature selection can improve performance, reduce training time, and help avoid overfitting.
- ❖ By reducing the number of input variables, feature selection helps to build simpler, faster, and more accurate models.

Why is Feature Selection Important for ANNs?

- ❖ Neural networks, especially deep ones, are powerful at finding complex patterns, but they can be sensitive to the quality of the input data. Including irrelevant or redundant features can lead to several problems:
- ❖ **Overfitting:** A model that is too complex might memorize the training data, including the noise, and perform poorly on new, unseen data. Irrelevant features add noise, making the model more likely to overfit.
- ❖ **Curse of Dimensionality:** As the number of features increases, the amount of data required to achieve a good model performance grows exponentially. This phenomenon, known as the curse of dimensionality, can make training a model with a large number of features computationally expensive and difficult.
- ❖ **Increased Training Time:** More features mean more weights and biases in the neural network, which increases the number of computations required during training. This can significantly slow down the training process.

- ❖ **Reduced Interpretability:** ANNs are often considered "black boxes." While it's already hard to understand how they make decisions, having fewer input features can make it easier to interpret the model's outputs and understand which factors are most influential.

Methods of Feature Selection in ANN

- ❖ Feature selection techniques can be broadly categorized into three types
- ❖ **Filter Methods:** These methods select features based on their statistical properties without involving a specific machine learning algorithm. They are generally computationally cheaper and faster than other methods. The chosen features are independent of the model.
 - ❖ **Correlation:** Features with a high correlation with the target variable are often considered more relevant.
 - ❖ **ANOVA (Analysis of Variance):** A statistical test that determines if there is a significant difference between the means of two or more groups, which can be used to assess the relationship between a feature and the target variable.
 - ❖ **Chi-squared test:** Used to test for the relationship between categorical variables.
 - ❖ **Mutual Information:** Measures the dependency between two variables. It can capture non-linear relationships, unlike correlation.
- ❖ **Wrapper Methods:** These methods use a specific machine learning model (in this case, an ANN) to evaluate subsets of features. They train a model on different feature subsets and select the one that results in the best performance (e.g., highest accuracy). While they can produce better results than filter methods, they are computationally more expensive.

- ❖ **Forward Selection:** Starts with an empty set of features and iteratively adds the feature that provides the greatest improvement to the model's performance until a stopping criterion is met.
- ❖ **Backward Elimination:** Starts with all features and iteratively removes the least significant feature until a stopping criterion is met.
- ❖ **Recursive Feature Elimination (RFE):** A greedy algorithm that repeatedly builds a model, ranks features based on their importance, and removes the least important ones. This process is repeated until the desired number of features is reached.
- ❖ **Embedded Methods:** These methods perform feature selection as part of the model training process itself. They combine the advantages of filter and wrapper methods by having a built-in mechanism for feature importance.
 - ❖ **Regularization:** Techniques like Lasso (L1 regularization) can be incorporated into the ANN's loss function. Lasso regularization adds a penalty equal to the absolute value of the magnitude of the weights. This penalty can force the weights of less important features to become zero, effectively removing them from the model.
 - ❖ **Tree-based methods:** While not strictly part of an ANN, algorithms like Random Forests and Gradient Boosting Machines can be used to pre-select features. These algorithms have a built-in feature importance score (e.g., Gini impurity or information gain) that can be used to rank and select features before training an ANN.
 - ❖ **LIME (Local Interpretable Model-agnostic Explanations) or SHAP (SHapley Additive exPlanations):** These are techniques for explaining the predictions of any

machine learning model. They can be used to understand which features have the most influence on an ANN's output, thus providing a basis for feature selection.

Feature Selection Techniques for ANN

❖ Using Feature Importance:

- ❖ Feature importance helps identify which input features are most useful for predicting the output. By using models like Random Forest or XGBoost, you can rank features based on how much they contribute to decision-making. Then, you can select the top-ranked features and feed them into your Artificial Neural Network (ANN) to improve performance and reduce complexity.

❖ Using Correlation Matrix:

- ❖ When building an Artificial Neural Network (ANN), it's important **to avoid redundant features that carry similar information**. One common way to detect and remove such redundancy is by using a correlation matrix.

❖ Using Autoencoders:

- ❖ Autoencoders are a type of neural network used to compress and reconstruct data. You can use them to automatically learn the most important features (called latent features), and then feed those compressed features into another model like an ANN for classification or prediction.

PCA (Principal Component Analysis):

- ❖ Principal Component Analysis (PCA) is a powerful technique used to reduce the number of input features in your dataset by transforming them into a smaller set of new features (principal components) that still capture the most important patterns (variance) in the data.

- ❖ Although PCA doesn't select the best original features (like feature selection), it transforms them into a smaller, more useful set — which can be used as input to an Artificial Neural Network (ANN).